

COMP9414 Artificial Intelligence

Topic exercises: Search with ai9414

UNSW Sydney

1 Instructions

These exercises practise the same ideas as the graph-search tutorial: DFS, BFS, uniform-cost search, and A* search.

Exercises 1 and 2 are intended to be worked through by hand, using the map, edge costs, heuristic table, and tie-breaking rule below. The later exercises use the ai9414 demos to inspect the same search ideas interactively.

Tie-breaking convention. When two frontier entries have the same priority, choose the node whose name comes first alphabetically. Unless the question says otherwise, stop when the goal node is expanded, not when it is first generated.

2 Exercise 1: Romania Route Finding

This exercise uses the route-finding map of Romania from Russell and Norvig. The task is to search for a route from **Arad** to **Bucharest**. Treat each city as a state and each road as an undirected weighted edge.

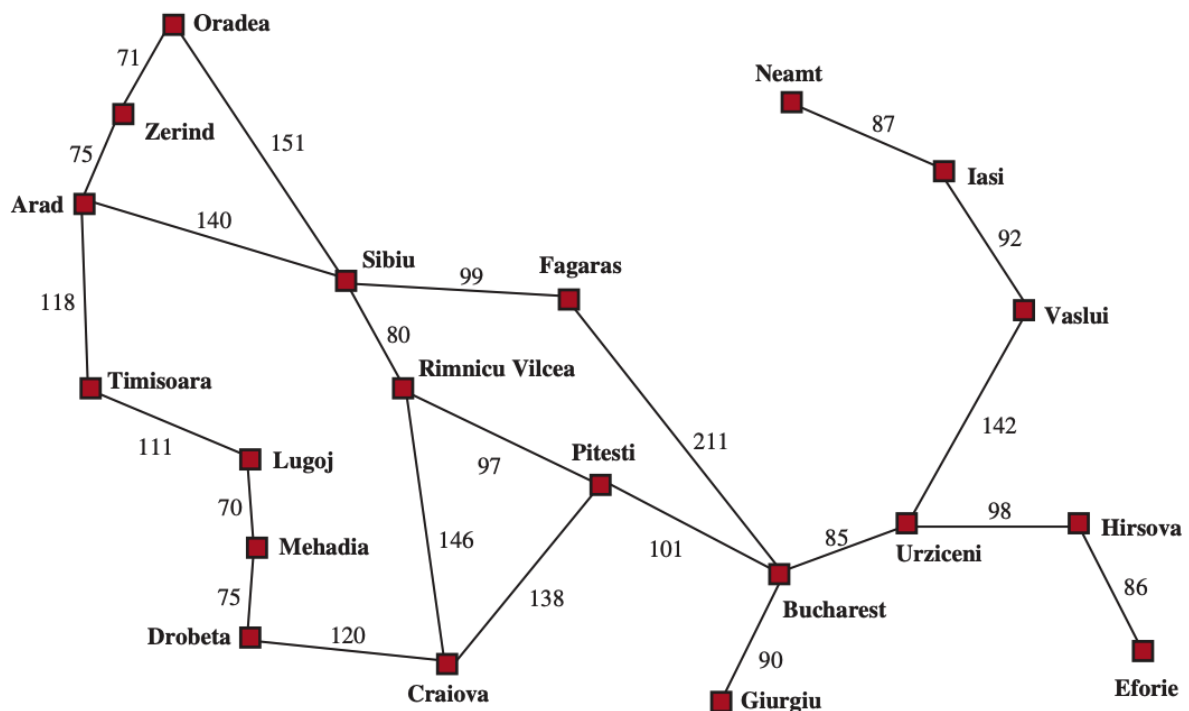


Figure 1: Romania route-finding map. Edge labels are road distances.

For A*, use the straight-line distance to Bucharest as the heuristic $h(n)$:

City	$h(n)$	City	$h(n)$	City	$h(n)$	City	$h(n)$
Arad	366	Fagaras	178	Lugoj	244	Sibiu	253
Bucharest	0	Giurgiu	77	Mehadia	241	Timisoara	329
Craiova	160	Hirsova	151	Neamt	234	Urziceni	80
Drobeta	242	Iasi	226	Oradea	380	Vaslui	199
Eforie	161	Pitesti	98	Rimnicu Vilcea	193	Zerind	374

Table 1: Straight-line distance heuristic to Bucharest.

1. Work out the order in which states are expanded by DFS. Use alphabetical neighbour order.
2. Work out the order in which states are expanded by BFS.
3. Work out the order in which states are expanded by UCS.
4. Work out the order in which states are expanded by A*.
5. Check the final route and total route cost for UCS and A*.
6. Decide which algorithms are guaranteed to return a lowest-cost route on this weighted graph, and explain why.

3 Exercise 2: A* Heuristic Perturbation

Keep the Romania map and the Arad-to-Bucharest problem from Exercise 1. Now change only two heuristic values:

$$h(\text{Fagaras}) = 176, \quad h(\text{Pitesti}) = 100.$$

1. Re-run A* by hand using the modified heuristic values.
2. Check whether the expansion order changes.
3. Check whether the returned path changes.
4. Decide whether the modified heuristic values are still admissible, and explain why.

4 Exercise 3: Comparing DFS and BFS in ai9414

Run the small graph DFS and BFS demos:

```
ai9414 demo graph-dfs --example small
ai9414 demo graph-bfs --example small
```

Use playback mode first, then use live-Python mode if you have implemented a Python solver.

1. Identify the start and goal labels used by both demos.
2. For DFS, inspect the returned path and visited order.

3. Identify one point where DFS chooses a branch that BFS does not choose as part of its final path.
4. For BFS, inspect the returned path and visited order.
5. Compare the number of edges in the DFS and BFS paths.
6. Discuss why BFS has a shallowest-path guarantee here, while DFS does not.

5 Exercise 4: Delivery as State-Space Search

The delivery demo is also a search problem, but the state is a grid position rather than a named graph node. Run:

```
ai9414 demo delivery --example four_rooms
```

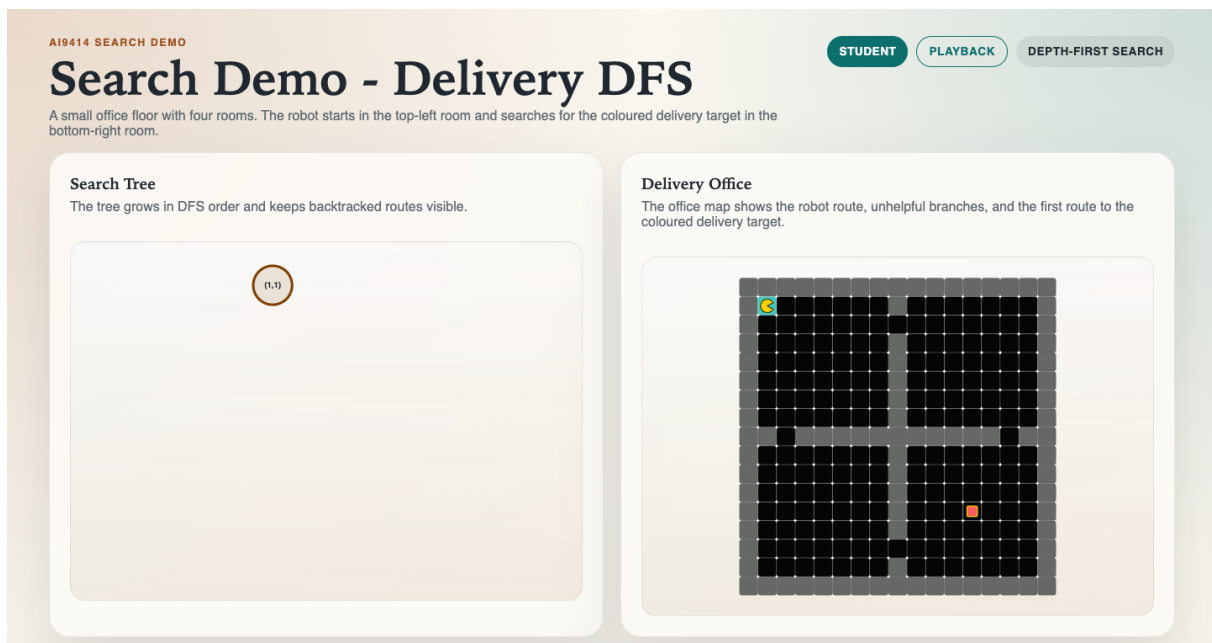


Figure 2: The `four_rooms` delivery example in playback mode.

1. Identify the start and target cells in the four-room example.
2. Run the playback and find where DFS backtracks.
3. Change the action-order control and run the demo again. How does the route change?
4. Compare the graph view and the search-tree view. Why does the target search avoid adding the same grid cell to the search tree twice?
5. Run the single-room example. Why is this case less useful for studying backtracking?

```
ai9414 demo delivery --example single_room
```

6 Exercise 5: Live-Python Replay

Choose one of the four graph-search demos and run the matching solution or your own implementation. The intended order is:

1. Start the demo.
2. In the browser, switch `Mode` to `live Python`.
3. Start the matching `solve_graph.py` file in a second terminal.
4. Click `Solve with Python`.
5. Click `Play`.

Things to do:

1. Try one graph-search demo with live Python enabled.
2. Use the replay to check that the browser follows your Python solver's trace.
3. Compare the final path from the replay with the path printed by your Python solver.
4. Identify which trace fields were needed for your chosen algorithm. Here, trace fields means the pieces of information your Python solver sends for each replay step, such as the current node, parent node, current path, cost, frontier priority, or considered edge.